

1) DataFrame Basics

1.1 Reading CSV (default)

```
people_df = spark.read.csv("file:///path/people.csv")
```

What it does: Reads CSV with default options:

- no header
- all columns as string
- comma delimiter

1.2 Reading CSV with options (recommended)

```
people_df = (  
    spark.read  
    .option("header", "true")  
    .option("inferSchema", "true")  
    .option("sep", ",")  
    .csv("file:///path/people.csv")  
)
```

Explanation

- `header=true` → first row becomes column names
- `inferSchema=true` → Spark detects numeric/date types
- `sep` → delimiter (, | \t etc.)

1.3 Print schema

```
people_df.printSchema()
```

Shows column names + data types.

1.4 Show data

```
people_df.show()          # default 20 rows
people_df.show(5)         # show 5 rows
people_df.show(truncate=False) # do not truncate long strings
```

1.5 Count rows

```
people_df.count()
```

1.6 Number of partitions

```
people_df.rdd.getNumPartitions()
```

2) RDD → DataFrame (Used in training, usually avoided in assessments)

2.1 Read as RDD and split

```
lines = sc.textFile("file:///path/people.txt")
parts = lines.map(lambda l: l.split(","))
```

2.2 Convert RDD to DataFrame using Row

```
from pyspark.sql import Row

people_rdd = parts.map(lambda p: Row(name=p[0], age=int(p[1])))
people_df = spark.createDataFrame(people_rdd)
people_df.show()
```

✓ **Note for assessments:** your anti-cheat often blocks `createDataFrame()` outside loader functions.

3) Temporary Views, SQL, Global Views

3.1 Temp view (session-scoped)

```
salesDF.createOrReplaceTempView("sales")
spark.sql("""
    SELECT Country, Category, SUM(Profit) AS total_profit
    FROM sales
    GROUP BY Country, Category
""").show()
```

3.2 Global view (cross-session)

```
salesDF.createOrReplaceGlobalTempView("gv_sales")
spark.sql("SELECT * FROM global_temp.gv_sales").show()
```

4) Core DataFrame Operations (Most used in assessments)

4.1 select / selectExpr

```
salesDF.select("Region", "Country").show()
```

```
totsales = salesDF.selectExpr(
    "Region",
    "Country",
    "State",
    "Category",
    "CAST(Profit AS BIGINT) AS Profit"
)
```

Why useful: `selectExpr()` lets you write SQL-like expressions.

4.2 filter / where

```
salesDF.filter("Country = 'India']").show()
salesDF.where(F.col("Country") == "India").show()
```

4.3 orderBy / sort

```
salesDF.filter("Country='India']").orderBy("Region").show()  
salesDF.filter("Country='India']").orderBy(F.col("Region").desc()).show()  
ow()
```

```
# sort is same as orderBy  
salesDF.filter("Country='India']").sort(F.col("Region")).show()
```

4.4 groupBy + agg

```
from pyspark.sql import functions as F  
  
salesDF.groupBy("Country", "Category") \  
    .agg(F.sum("Profit").alias("totProfit")) \  
    .show()
```

5) JSON (Including Nested JSON)

5.1 Read JSON

```
df_json = spark.read.option("inferSchema",  
"true").json("file:///path/employees.json")  
df_json.printSchema()  
df_json.show()
```

5.2 Nested JSON access

Example record:

```
{"col1":{"col2":"2","col3":["arr1","arr2"]}}
```

PySpark:

```
jsondf = spark.read.json("nested.json")  
  
jsondf.select("col1.col2").show()  
jsondf.select(F.col("col1.col3")[0].alias("first_item")).show()
```

SQL way:

```
jsondf.createOrReplaceTempView("jsonnested")
spark.sql("SELECT col1.col3[0] AS first_item FROM
jsonnested").show()
```

6) Avro (Spark doesn't always ship Avro by default)

6.1 Read Avro (Spark 3.x usually supports with package)

```
avro_df = spark.read.format("avro").load("file:///path/users.avro")
```

If package needed:

```
pyspark --packages org.apache.spark:spark-avro_2.12:<spark_version>
```

7) Writing DataFrames (CSV/Parquet/ORC)

7.1 Write CSV

```
salesDF.write.mode("overwrite").option("header",
"true").csv("file:///path/out_csv")
```

7.2 Write Parquet

```
salesDF.write.mode("overwrite").parquet("file:///path/out_parquet")
```

7.3 Read Parquet

```
sales_par = spark.read.parquet("file:///path/out_parquet")
```

7.4 Write ORC

```
salesDF.write.mode("overwrite").format("orc").save("file:///path/out_orc")
```

8) Hive Tables (If Hive support enabled)

8.1 Save as table

```
spark.sql("USE mydb")
salesDF.write.mode("overwrite").saveAsTable("salesdf")
spark.sql("SHOW TABLES").show()
```

8.2 Read table as DataFrame

```
newsalesDF = spark.read.table("mydb.salesdf")
```

9) Broadcast Joins (Performance + often tested)

9.1 Broadcast a small dimension table

```
from pyspark.sql.functions import broadcast

joined = bigDF.join(broadcast(smallDF), "id", "inner")
```

Explanation: Forces Spark to ship smallDF to all executors and avoid shuffle.

9.2 Auto broadcast threshold

```
spark.conf.get("spark.sql.autoBroadcastJoinThreshold")
```

```
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 10485760) #
bytes
```

10) Joins (All Join Types)

```
empDF.join(deptDF, empDF.emp_dept_id == deptDF.dept_id,
"inner").show()
empDF.join(deptDF, empDF.emp_dept_id == deptDF.dept_id,
"left").show()
empDF.join(deptDF, empDF.emp_dept_id == deptDF.dept_id,
"right").show()
empDF.join(deptDF, empDF.emp_dept_id == deptDF.dept_id,
"outer").show()
```

```
# leftanti: rows in empDF with no match in deptDF
empDF.join(deptDF, empDF.emp_dept_id == deptDF.dept_id,
"leftanti").show()
```

```
# leftsemi: rows in empDF that have a match in deptDF
empDF.join(deptDF, empDF.emp_dept_id == deptDF.dept_id,
"leftsemi").show()
```

11) Schemas (StructType / StructField)

11.1 Create schema and DataFrame

```
from pyspark.sql.types import StructType, StructField, IntegerType,
StringType
```

```
emp_schema = StructType([
    StructField("Empno", IntegerType(), True),
    StructField("Empname", StringType(), True),
    StructField("Location", StringType(), True),
])
```

```
emp_data = [(1, "A", "Chennai"), (2, "B", "Mumbai"), (3, "C",  
"Bengaluru")]  
empDF = spark.createDataFrame(emp_data, schema=emp_schema)  
  
empDF.printSchema()  
empDF.show(truncate=False)
```

12) Bad Record Handling (CSV read modes)

```
empDF = spark.read.schema(emp_schema).option("mode",  
"DROPMALFORMED").csv("file:///path/emp.csv")  
empDF = spark.read.schema(emp_schema).option("mode",  
"FAILFAST").csv("file:///path/emp.csv")  
empDF = spark.read.schema(emp_schema).option("mode",  
"PERMISSIVE").csv("file:///path/emp.csv")
```

Modes

- **DROPMALFORMED**: skip bad rows
 - **FAILFAST**: stop immediately
 - **PERMISSIVE**: keep row, invalid fields become null
-

13) Compression (Common in writing)

13.1 CSV compression

```
salesDF.write.mode("overwrite").option("codec",  
"bzip2").csv("file:///path/out_csv_bzip2")
```

13.2 Parquet compression

```
spark.conf.set("spark.sql.parquet.compression.codec", "gzip")
salesDF.write.mode("overwrite").parquet("file:///path/out_parquet_gzip")
```

Common codecs:

- `snappy` (default parquet)
 - `gzip`
 - `lz4`
 - `bzip2`
-

14) JDBC Read (MySQL)

14.1 Read table

```
df_mysql = (
    spark.read.format("jdbc")
        .option("url",
            "jdbc:mysql://localhost:3306/retail?useSSL=false&allowPublicKeyRetrieval=true")
        .option("driver", "com.mysql.jdbc.Driver")
        .option("dbtable", "order_items")
        .option("user", "ak")
        .option("password", "ak")
        .load()
)
```

14.2 Partitioned read (parallelism)

```
df_mysql = (
    spark.read.format("jdbc")
        .option("url",
            "jdbc:mysql://localhost:3306/retail?useSSL=false&allowPublicKeyRetrieval=true")
```

```

.option("driver", "com.mysql.jdbc.Driver")
.option("dbtable", "order_items")
.option("partitionColumn", "order_item_id")
.option("lowerBound", "1")
.option("upperBound", "172168")
.option("numPartitions", "10")
.option("user", "ak")
.option("password", "ak")
.load()
)

```

14.3 Predicate pushdown using query/dbtable alias

```

q = "(select * from order_items where order_item_order_id > 100) t"
df_mysql = spark.read.format("jdbc") \
    .option("url",
"jdbc:mysql://localhost:3306/retail?useSSL=false&allowPublicKeyRetri
eval=true") \
    .option("driver", "com.mysql.jdbc.Driver") \
    .option("dbtable", q) \
    .option("user", "ak") \
    .option("password", "ak") \
    .load()

```

15) Additional PySpark Functions (Highly Useful in Assessments)

15.1 withColumn

```
df2 = df.withColumn("profit2x", F.col("Profit") * 2)
```

15.2 when / otherwise

```

df2 = df.withColumn(
    "profit_flag",
    F.when(F.col("Profit") > 0,
F.lit("POS")).otherwise(F.lit("NEG"))
)

```

)

15.3 coalesce (fill nulls)

```
df2 = df.withColumn("full_name", F.coalesce("full_name",  
F.concat_ws(" ", "first_name", "last_name")))
```

15.4 dropDuplicates

```
df_unique = df.dropDuplicates(["user_id"])
```

15.5 unionByName

```
df_all = df1.unionByName(df2)
```

15.6 distinct

```
df.select("Country").distinct().show()
```

15.7 explode (arrays)

```
df2 = df.select("id", F.explode("items").alias("item"))
```

15.8 split (strings)

```
df2 = df.withColumn("parts", F.split(F.col("name"), " "))
```

15.9 window functions (ranking/top N)

```
from pyspark.sql.window import Window
```

```
w = Window.partitionBy("Country").orderBy(F.col("Profit").desc())  
df2 = df.withColumn("rn", F.row_number().over(w)).filter(F.col("rn")  
<= 3)
```

SET 1: Big Data Fundamentals

Q1. Which factor best differentiates Big Data from traditional data systems?

- A. Data is always numeric
- B. Data size alone
- C. Need for distributed storage and processing
- D. Use of SQL

✅ **Correct Answer: C**

Explanation:

Big Data is not just “large data” — it requires **distributed storage + parallel processing**.

- Size alone is insufficient
 - SQL can still be used
 - Numeric vs non-numeric is irrelevant
-

Q2. Which scenario best represents *Variety* in Big Data?

- A. A 10TB relational database
- B. Real-time stock price updates
- C. Text, images, videos, and JSON logs together
- D. Incorrect data values

✅ **Correct Answer: C**

Explanation:

- **Variety** = multiple formats
 - A = Volume
 - B = Velocity
 - D = Veracity
-

Q3. Why is unstructured data harder to process?

- A. It is always encrypted
- B. It lacks a predefined schema
- C. It is too small
- D. It cannot be distributed

✓ **Correct Answer: B**

Explanation:

Unstructured data does not follow rows/columns → needs schema-on-read processing.

◆ SET 2: Distributed Computing & Hadoop

Q4. Why do distributed systems use horizontal scaling?

- A. To reduce SQL usage
- B. To increase clock speed
- C. To add more machines instead of bigger machines
- D. To avoid networking

✓ **Correct Answer: C**

Explanation:

Big Data systems grow by **adding nodes**, not upgrading one server.

Q5. Which Hadoop component manages cluster resources?

- A. HDFS
- B. MapReduce
- C. YARN
- D. Hive

✓ **Correct Answer: C**

Explanation:

- HDFS → storage
 - MapReduce → compute
 - **YARN → resource management**
-

Q6. Which daemon actually stores data blocks in HDFS?

- A. NameNode
- B. ResourceManager
- C. NodeManager
- D. DataNode

✓ **Correct Answer: D**

Explanation:

NameNode stores metadata; **DataNode stores real data.**

◆ SET 3: Apache Spark Overview

Q7. Apache Spark is best described as:

- A. A distributed database
- B. A file system
- C. A distributed data processing engine
- D. A visualization tool

✓ **Correct Answer: C**

Explanation:

Spark processes data; it doesn't store it permanently.

Q8. Why is Spark faster than MapReduce?

- A. It uses GPUs
- B. It avoids disk I/O using in-memory computation
- C. It uses SQL only
- D. It compresses files automatically

✓ **Correct Answer: B**

Explanation:

Spark keeps intermediate data in memory → huge speed improvement.

Q9. Which belongs to the Spark ecosystem?

- A. MySQL
- B. Spark SQL
- C. Tableau
- D. MongoDB

✓ Correct Answer: B

◆ SET 4: Spark Architecture (Very Important)

Q10. Which component schedules tasks and builds the execution DAG?

- A. Executor
- B. Worker
- C. Cluster Manager
- D. Driver

✓ Correct Answer: D

Explanation:

The **Driver** controls execution logic.

Q11. Which component executes tasks?

- A. Driver
- B. Executor
- C. DAG Scheduler
- D. NameNode

✓ Correct Answer: B

Q12. Which component allocates resources to Spark applications?

- A. Executor
- B. Driver
- C. Cluster Manager
- D. SparkSession

✓ Correct Answer: C

◆ SET 5: Spark Execution Model (Top Exam Area)

Q13. What triggers execution in Spark?

- A. Creating an RDD
- B. Applying a transformation

- C. Calling an action
- D. Defining a function

✓ **Correct Answer: C**

Explanation:

Spark is **lazy** — only actions trigger computation.

Q14. Correct execution flow is:

- A. Tasks → Jobs → DAG
- B. User Code → DAG → Stages → Tasks → Executors
- C. DAG → Tasks → Jobs
- D. Executors → Tasks → DAG

✓ **Correct Answer: B**

Q15. Which operation causes a shuffle?

- A. map()
- B. filter()
- C. groupBy()
- D. select()

✓ **Correct Answer: C**

Explanation:

groupBy requires data movement across partitions → **wide transformation**.

◆ **SET 6: RDD Concepts**

Q16. RDD stands for:

- A. Relational Distributed Data
- B. Resilient Distributed Dataset
- C. Random Data Directory
- D. Remote Data Driver

✓ **Correct Answer: B**

Q17. Why are RDDs fault tolerant?

- A. They replicate all data
- B. They store backups in MySQL
- C. They recompute lost partitions using lineage
- D. They use checkpoints only

✓ **Correct Answer: C**

Q18. Which creates an RDD from a Python list?

- A. `spark.rdd([1,2,3])`
- B. `rdd.create()`
- C. `df.toRDD()`
- D. `sc.parallelize([1,2,3])`

✓ **Correct Answer: D**

◆ **SET 7: Transformations vs Actions**

Q19. Which is a transformation?

- A. `count()`
- B. `collect()`
- C. `filter()`
- D. `show()`

✓ **Correct Answer: C**

Q20. Which is an action?

- A. `map()`
- B. `reduce()`
- C. `filter()`
- D. `flatMap()`

✓ **Correct Answer: B**

Q21. Nothing appears in Spark UI after many transformations. Why?

- A. Cluster failed
- B. Missing action

- C. No executor
- D. Wrong syntax

✓ **Correct Answer: B**

◆ SET 8: DataFrames & Spark SQL

Q22. What does `spark.sql()` return?

- A. List
- B. Pandas DataFrame
- C. Spark DataFrame
- D. CSV

✓ **Correct Answer: C**

Q23. Which format is columnar and optimized for analytics?

- A. CSV
- B. JSON
- C. Parquet
- D. TXT

✓ **Correct Answer: C**

Q24. Which enables automatic datatype detection?

- A. `inferSchema=True`
- B. `header=True`
- C. `schema=None`
- D. `cast()`

✓ **Correct Answer: A**

◆ SET 9: Performance & Optimization

Q25. What is the role of Catalyst Optimizer?

- A. Cluster scaling
- B. SQL and DataFrame optimization

- C. Memory management
- D. Streaming control

✓ **Correct Answer: B**

Q26. Why are Python UDFs slower?

- A. Use disk
- B. Disable caching
- C. Bypass Catalyst + Python-JVM overhead
- D. Require shuffles

✓ **Correct Answer: C**

Q27. Job slowed after adding groupBy(). Why?

- A. Driver crash
- B. Memory leak
- C. Shuffle occurred
- D. Cache disabled

✓ **Correct Answer: C**

◆ SET 10: Streaming & Fault Tolerance

Q28. What enables recovery after streaming failure?

- A. Replication
- B. Checkpointing
- C. Caching
- D. Repartitioning

✓ **Correct Answer: B**

Q29. How does Spark handle late data?

- A. Drops it
- B. Reprocesses everything
- C. Uses watermarking
- D. Uses caching

✓ **Correct Answer: C**

Q30. Kafka producing faster than Spark consuming — solution?

- A. Increase batch size
- B. Disable checkpointing
- C. Enable backpressure
- D. Stop Kafka

✓ **Correct Answer: C**

FINAL EXAM STRATEGY

- **Recompute after failure?** → Lineage
- **Nothing executes?** → Missing action
- **Slow after groupBy/join?** → Shuffle
- **Streaming restart safe?** → Checkpointing
- **Rolling totals?** → window + slide